

План действий: WindSpotBot

Общий срок

- **Ориентировочно**: 10-14 недель (21 апреля – 31 июля 2025).
- **Гибкость**: Сроки корректируются в зависимости от времени заказчика.
- **Контроль**: Еженедельные отчёты, согласование перехода между этапами.

Этапы

Этап 1: Подготовка и проектирование (1-2 недели, 21 апреля – 4 мая 2025)

Цель: Создать структуру проекта, настроить окружение, спроектировать сервисы и репозитории.

Задачи:

1. Создать публичный репозиторий на GitHub:
 - Ветки: `main`, `dev`, `feature/\*`.
 - `.gitignore` для Python, `.env`, логов.
2. Настроить CI/CD (GitHub Actions):

```
```yaml
name: CI
on: [push, pull_request]
jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v3
 - run: pip install -r requirements.txt
 - run: pytest --cov=src --cov-report=xml
 - run: flake8 src
 - run: black --check src
```
```

3. Создать структуру проекта:

```
```
src/
 handlers/
 checkin.py
 spots.py
 weather.py
 profile.py
 admin.py
 chat.py
 services/
 checkin.py
 spot.py
 weather.py
 notification.py
 chat.py
 rating.py
 stats.py
 scheduler.py
 repositories/
 user.py
 spot.py
 checkin.py
 models/
 user.py
 spot.py
 checkin.py
 keyboards.py
 config.py
 bot.py
 tests/
 test_checkin.py
 test_spot.py
 migrations/
 env.py
```
```

```
docs/
  architecture.md
  ux.md
  api.md
README.md
CHANGELOG.md
.env.example
requirements.txt
Dockerfile
docker-compose.yml
```
```

4. Настроить конфигурацию:

```
```python
from pydantic import BaseSettings

class Settings(BaseSettings):
    BOT_TOKEN: str
    CHAT_ID: str
    ADMINS: List[int]
    NEARBY_SPOTS_LIMIT: int = 5
    PLANNED_VISITS_LIMIT: int = 5
    CHECKIN_CHECK_INTERVAL_MINUTES: int = 5
    WEATHER_CHECK_INTERVAL_MINUTES: int = 15
    MESSAGE_TTL_DAYS: int = 7
    STATS_CACHE_TTL_SECONDS: int = 600

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"

settings = Settings()
```
```

5. Настроить логирование:

```
```python
import logging.config

LOGGING_CONFIG = {
    "version": 1,
    "formatters": {
        "default": {"format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s"}
    },
    "handlers": {
        "console": {"class": "logging.StreamHandler", "formatter": "default"},
        "file": {
            "class": "logging.handlers.RotatingFileHandler",
            "formatter": "default",
            "filename": "bot.log",
            "maxBytes": 10485760,
            "backupCount": 5,
        },
    },
    "loggers": {
        "bot": {"level": "DEBUG", "handlers": ["console", "file"]}
    },
}
logging.config.dictConfig(LOGGING_CONFIG)
```
```

6. Настроить `alembic`:

```
```bash
alembic init migrations
- Создать миграцию для `users`, `spots`, `checkins`, `spot_ratings`.
```

7. Реализовать `keyboards.py`:

```
```python
from aiogram.types import InlineKeyboardMarkup, InlineKeyboardButton

def get_checkin_duration_keyboard() -> InlineKeyboardMarkup:
 """Клавиатура для выбора длительности чек-ина."""
 keyboard = InlineKeyboardMarkup(row_width=3)
 durations = ["1 час", "2 часа", "3 часа", "4 часа", "5 часов", "6 часов"]
 buttons = [InlineKeyboardButton(text=d, callback_data=f"duration:{d}")
 for d in durations]
 keyboard.add(*buttons)
 keyboard.add(InlineKeyboardButton(text="⬅️ Назад", callback_data="back"))
 return keyboard
```
```

8. Спроектировать сервисы и репозитории:

- Сервисы: `CheckinService`, `SpotService`, `NotificationService`, `ChatService`, `RatingService`, `WeatherService`, `SchedulerService`, `StatsService`.
- Репозитории: `UserRepository`, `SpotRepository`, `CheckinRepository`.

9. Написать документацию:

- `README.md`: установка, запуск, зависимости.
- `docs/architecture.md`: описание сервисов, репозиториев.
- `docs/ux.md`: морской тон, эмодзи.
- `docs/api.md`: план API.

****Критерии завершения**:**

- Репозиторий создан, CI/CD работает.
- Структура проекта реализована.
- `config.py`, `keyboards.py`, `bot.py` готовы.
- `alembic` настроен, созданы начальные миграции.
- Документация (`README.md`, `docs/\*`) готова.
- Сервисы и репозитории спроектированы.

****Контроль**:**

- Отчёт 28 апреля 2025: «Завершено 50% Этапа 1».
- Согласование 4 мая 2025: «Этап 1 готов, подтверждаешь переход к Этапу 2?».

Этап 2: Базовый функционал (3-4 недели, 5 мая – 2 июня 2025)

****Цель**:** Реализовать функции высокого приоритета: чек-ины, «Кто на спотах», уведомления, страничка спота, оценка, хэштег-чат.

****Задачи**:**

1. Реализовать сервисы:

- `CheckinService`, `SpotService`, `NotificationService`, `ChatService`, `RatingService`.

2. Реализовать репозитории:

- `UserRepository`, `SpotRepository`, `CheckinRepository`.

3. Реализовать хендлеры:

- `/start`, чек-ины, «Кто на спотах», страничка спота, оценка, чат.

4. Настроить кэширование (Вариант 1):

```
```python
@cached(ttl=3600, key_builder=lambda *args, **kwargs: f"nearby_spots:
{kwargs['lat']}:{kwargs['lon']}")
async def get_nearby_spots(self, lat: float, lon: float, limit: int) ->
List[dict]:
 """Получение ближайших спотов (без активных пользователей)."""
```
```

5. Написать тесты:

```
```python
async def test_create_checkin(mocked):
 mocker.patch("src.repositories.checkin.CheckinRepository.create",
return_value=1)
 checkin_id = await CheckinService().create_checkin(123, 1, 1, 3600)
 assert checkin_id == 1
```
```

6. Провести нагрузочное тестирование хэштег-чата (1000 сообщений).
7. Интегрировать `@WindSpotRU`.

****Критерии завершения**:**

- Реализованы функции высокого приоритета.
- Тесты покрывают >50% кода.
- Хэштег-чат стабилен.
- Уведомления работают.

****Контроль**:**

- Отчёты: 19 мая, 26 мая.
- Демо: 2 июня.
- Согласование перехода к Этапу 3.

Этап 3: Средний приоритет (2-3 недели, 3 июня – 23 июня 2025)

****Цель**:** Реализовать функции среднего приоритета: планирование, погода, профиль, геймификация, социальные функции.

****Задачи**:**

1. Реализовать `WeatherService`, `SchedulerService`.
2. Реализовать геймификацию и социальные функции.
3. Провести А/В-тестирование клавиатур.
4. Собрать фидбэк от 5-10 пользователей.
5. Написать тесты.

****Критерии завершения**:**

- Реализованы функции среднего приоритета.
- Тесты покрывают >70% кода.
- А/В-тестирование завершено.
- Фидбэк собран.

****Контроль**:**

- Отчёты: 16 июня.
- Презентация фидбэка: 23 июня.
- Согласование перехода к Этапу 4.

Этап 4: Низкий приоритет и релиз (2 недели, 24 июня – 7 июля 2025)

****Цель**:** Реализовать функции низкого приоритета, протестировать, выпустить v1.0.0.

****Задачи**:**

1. Реализовать `StatsService`, `/stats`.
2. Реализовать рекомендации, настройку уведомлений.
3. Провести финальное тестирование.
4. Подготовить релиз:
 - Тег v1.0.0.
 - `CHANGELOG.md`.
5. Написать финальную документацию.

****Критерии завершения**:**

- Все функции реализованы.
- Тесты покрывают >80% кода.
- Релиз v1.0.0 опубликован.

****Контроль**:**

- Отчёт: 30 июня.
- Демо: 7 июля.
- Подтверждение релиза.

Риски

- ****Ограниченное время**:** Гибкие сроки, приоритизация задач.
- ****Ошибки в API**:** Моки для Open-Meteo, запасной API.
- ****Нагрузка на чат**:** Нагрузочное тестирование, запасные варианты (группы, база).